

Arquivo: cli.py

```
"""
DESCRIÇÃO GERAL:
Este é o ponto de interação com o usuário (Interface de Linha de Comando - CLI).
Ele interpreta o que o usuário digita no terminal (ex: "dfs put teste.txt docs/teste.txt"),
transforma isso em variáveis, chama a camada de cliente de rede e exibe o resultado
de forma visual na tela.
"""

# argparse é a biblioteca padrão do Python mais robusta para criar CLIs.
# Ela gera menus de ajuda (-h) automaticamente e lida com parâmetros obrigatórios/opcionais.
import argparse

# Path para manipular os arquivos locais do cliente (quando for fazer upload/download).
from pathlib import Path

# Função que esconde toda a complexidade da rede e apenas recebe os dados da operação.
from dfs.client import send_request


def build_parser() -> argparse.ArgumentParser:
    """
    Monta a interface de linha de comando.
    """
    # Cria o avaliador (parser) base, batizando o programa de "dfs".
    parser = argparse.ArgumentParser(prog="dfs")

    # Cria "subparsers" (subcomandos), como os usados no git (git clone, git pull, etc).
    # dest="command" salvará a palavra digitada na propriedade args.command.
    # required=True obriga o usuário a escolher uma opção.
    sub = parser.add_subparsers(dest="command", required=True)

    # Subcomando PUT.
    put = sub.add_parser("put", help="Envia um arquivo para o DFS")
    put.add_argument("source", help="arquivo local de origem") # Argumento obrigatório
    put.add_argument("target", help="caminho lógico no DFS") # Argumento obrigatório

    # Subcomando GET.
    get = sub.add_parser("get", help="Lê um arquivo do DFS")
    get.add_argument("path", help="caminho lógico no DFS") # Argumento obrigatório
    # nargs="?" significa que é opcional. Se não for passado, default será None.
    get.add_argument("output", nargs="?", default=None, help="arquivo local de saída")

    # Subcomando LIST (não tem argumentos adicionais, ele apenas roda).
    sub.add_parser("list", help="Lista arquivos no DFS")

    # Subcomando RM (Remove).
    rm = sub.add_parser("rm", help="Remove um arquivo do DFS")
    rm.add_argument("path", help="caminho lógico no DFS") # Argumento obrigatório

    return parser
```

```

def main(argv=None) -> None:
    """
    Executa o comando solicitado pelo usuário.
    """
    # Inicializa o parser e analisa os argumentos passados pelo terminal.
    # Se argv for None, sys.argv é capturado automaticamente pelo argparse.
    parser = build_parser()
    args = parser.parse_args(argv)

    # Bloco condicional para direcionar o subcomando recebido.
    if args.command == "put":
        # Path().read_bytes() lê o arquivo inteiro da máquina local do usuário para a memória RAM.
        data = Path(args.source).read_bytes()

        # Pede para o client de rede enviar o pacote para o servidor.
        response = send_request("PUT", path=args.target, data=data)

        # Imprime a mensagem retornada pelo servidor ("Arquivo salvo com sucesso").
        print(response.message)
        return

    if args.command == "get":
        # Faz a requisição de download para o servidor. Não precisa enviar data.
        response = send_request("GET", path=args.path)

        # Se a operação falhou (ex: arquivo não existe no DFS), imprime o erro e encerra o fluxo.
        if not response.ok:
            print(response.message)
            return

        # Define onde salvar localmente.
        # Usa o argumento passado ou, se não passado, usa o próprio nome original,
        # ou "saida.bin" como fallback final de emergência.
        output = args.output or Path(args.path).name or "saida.bin"

        # Escreve fisicamente no HD local do usuário os bytes retornados na resposta.
        Path(output).write_bytes(response.data)
        print(f"{response.message} -> salvo em {output}")
        return

    if args.command == "list":
        # Envia a requisição LIST.
        response = send_request("LIST")

        # Verifica falhas.
        if not response.ok:
            print(response.message)
            return

        # Se não houver itens na lista (vazio).
        if not response.entries:

```

```
        print("(vazio)")
        return

    # Itera sobre os caminhos devolvidos pelo servidor e os imprime um a um.
    for entry in response.entries:
        print(entry)
    return

if args.command == "rm":
    # Envia a instrução de exclusão ao servidor.
    response = send_request("DELETE", path=args.path)
    print(response.message)
    return

# Bloco padrão do Python para testar esse arquivo diretamente.
# Contudo, o ponto de entrada principal pretendido está no arquivo __main__.py
if __name__ == "__main__":
    main()
```